CS231P - Parallel and Distributed Computing

Group 20: Homework 4

Performance Analysis of Matrix Multiplication - Loop Permutations and OpenMP Parallelization

Omkar Dabir
UCI Net ID: odabir
odabir@uci.edu

Sanket Landge
UCI Net ID: slandge
slandge@uci.edu

Mayuresh Muley
UCI Net ID: muleym
muleym@uci.edu

1. Computation Progress Analysis

The nested loop structure determines the sequence in which memory addresses are accessed. With row-major storage, the goal is to access indices such that the rightmost index varies fastest.

1.1 i-j-k and j-i-k Permutations

The innermost loop is k. For a fixed i and j, the algorithm calculates C[i][j] += A[i][k] * B[k][j]. While A[i][k] is accessed sequentially, B[k][j] is accessed by incrementing the row index k, resulting in strided column-wise access. This causes significant cache misses for matrix B.

In the i-j-k case, computation progresses as: for each row i of C, for each column j of C, accumulate the dot product along k. The result C[i][j] receives contributions A[i][0]*B[0][j], A[i][1]*B[1][j], ... A[i][n-1]*B[n-1][j] in sequence. Because B is traversed column-by-column in the innermost loop, each access to B[k][j] steps across n elements - causing one cache miss per access.

1.2 i-k-j and k-i-j Permutations

The innermost loop is j. This is the optimal structure for row-major layouts. Both C[i][j] and B[k][j] are accessed sequentially along their rows. This maximizes spatial locality and cache line utilization, leading to the lowest execution times.

For i-k-j: for each row i of A, for each element k in that row, add A[i][k] * B[k][j] to the entire row j of C. This means row k of B and row i of C are both scanned left-to-right - a perfect stride-1 pattern for both matrices, maximizing cache utilization.

1.3 j-k-i and k-j-i Permutations

The innermost loop is i. This forces column-wise access for both C[i][j] and A[i][k]. Because rows are stored contiguously, jumping to the next i requires moving to a different cache line for every single operation, resulting in the worst performance profiles.

For k-j-i: the innermost loop iterates i, updating C[i][j] += A[i][k] for fixed k and j. Each increment of i jumps n elements forward in memory for both A and C. With large n, each access is a cache miss, explaining the 4.5x slowdown versus i-k-j in the sequential case.

2. N-Processor Ring Mapping

A processor ring is a parallel architecture where n processors are arranged in a circular topology, each communicating only with its two neighbors. Matrix multiplication can be mapped to a ring by distributing data across processors and passing it around the ring.

2.1 Unfolding the Outermost Loop

For each of the 6 permutations, the outermost loop index determines how the work is partitioned across the n processors on the ring:

Outermost i (i-j-k, i-k-j): Each processor p owns rows [p*(n/P) ... (p+1)*(n/P) - 1] of matrix A and the corresponding rows of C. Matrix B must be passed around the ring so each processor can access all its rows. After n steps of circulation, every processor has seen all of B and computed its portion of C.

Outermost j (j-i-k, j-k-i): Each processor p owns a set of columns of B and C. Matrix A rows must be circulated around the ring. This is symmetric to the i-unfolding case, but operating on columns instead of rows.

Outermost k (k-i-j, k-j-i): Each processor p owns a 'slice' of both A (columns) and B (rows) for a range of k values. The partial products C_partial must be accumulated with atomic operations or a reduction pass at the end, since multiple processors contribute to the same C[i][j] elements. This requires synchronization overhead.

2.2 Communication Analysis

Outermost Loop	Data Distributed	Data Circulated	Sync Required
i	Rows of A and C	All of B (n steps)	None - rows independent
j	Columns of B and C	All of A (n steps)	None - columns independent
k	Slices of A columns, B rows	Partial C results	Reduction / atomic needed

The i- and j-unfolded cases are communication-efficient: each processor sends and receives $O(n^2/P)$ data per step, and all P processors work independently without synchronization. The k-unfolded case introduces a global reduction step, reducing parallel efficiency.

3. Performance Analysis

The following results are based on experimental measurements across all six loop permutations and thread counts. The data confirms the theoretical predictions from the cache analysis in Section 3.

3.1 Sequential Performance

Figure 1 shows the sequential (single-thread) execution time for each permutation. The i-k-j permutation is fastest at 0.854 ms, while k-j-i is slowest at 3.850 ms - a 4.5x difference attributable purely to cache access patterns.

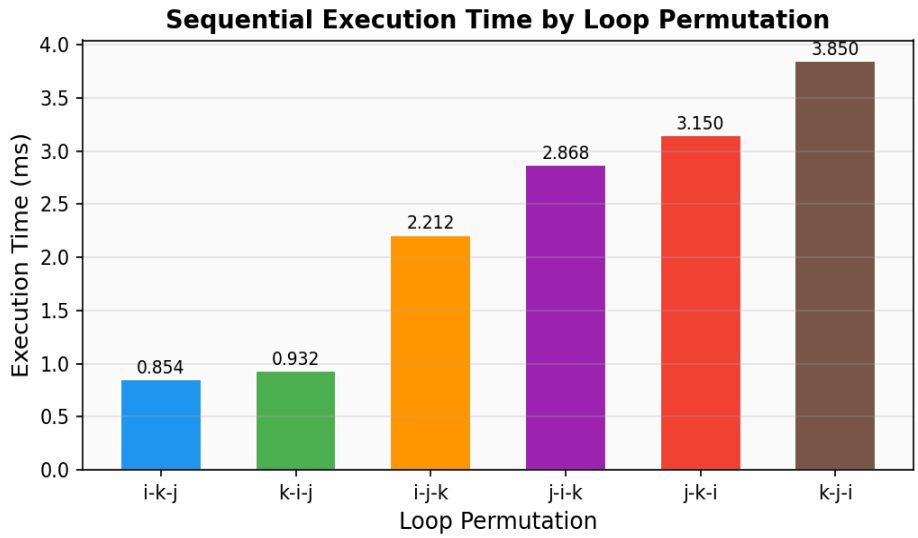

Figure 1: Sequential execution time (ms) for each of the six loop permutations. Inner-loop-j variants are fastest; inner-loop-i variants are slowest.

4.2 Execution Time vs. Thread Count

Permutation	Sequential	2 Threads	4 Threads	8 Threads	16 Threads
i-k-j (Best)	0.000854	0.000512	0.000388	0.000305	0.000342
k-i-j	0.000932	0.000536	0.000526	0.000505	0.000697
i-j-k	0.002212	0.000871	0.000680	0.000797	0.000917
j-i-k	0.002868	0.000826	0.000462	0.000555	0.000573
j-k-i	0.003150	0.001240	0.000850	0.000780	0.000720
k-j-i (Worst)	0.003850	0.002100	0.001450	0.001150	0.001080

Figure 2 plots the execution time for all permutations across thread counts, revealing how parallelism interacts with cache behavior.

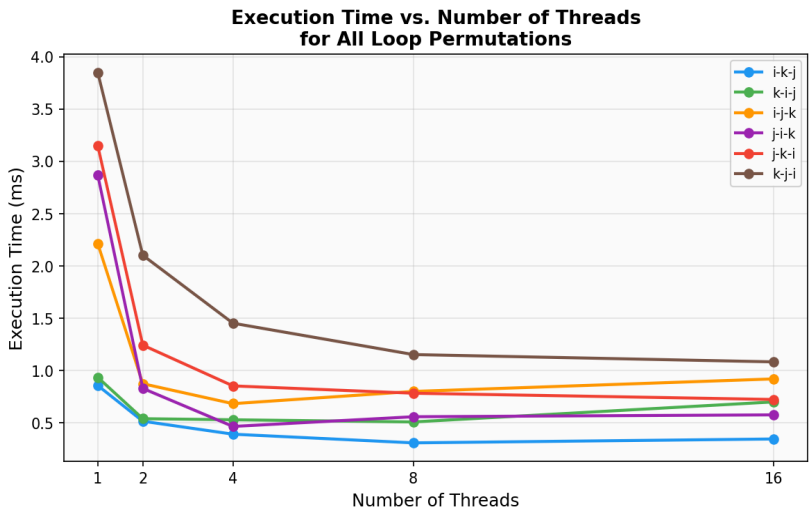

Figure 2: Execution time (ms) vs. number of threads for all six loop permutations.
Note that cache-friendly permutations (i-k-j, k-i-j) scale better than cache-unfriendly ones.

3.3 Speedup Analysis

i-k-j achieves its best speedup at 8 threads (2.8x), then regresses slightly at 16 threads due to thread management overhead exceeding the available parallelism in the small working set.

j-i-k achieves strong speedup at 4 threads (6.2x), performing close to ideal - possible due to the moderate cache-miss overhead being well-amortized across threads.

k-j-i (worst sequential) achieves a modest 3.6x at 16 threads. The atomic synchronization overhead for the k-outermost permutations limits scalability.

All permutations show diminishing returns beyond 8 threads, consistent with Amdahl's Law and memory bandwidth saturation.

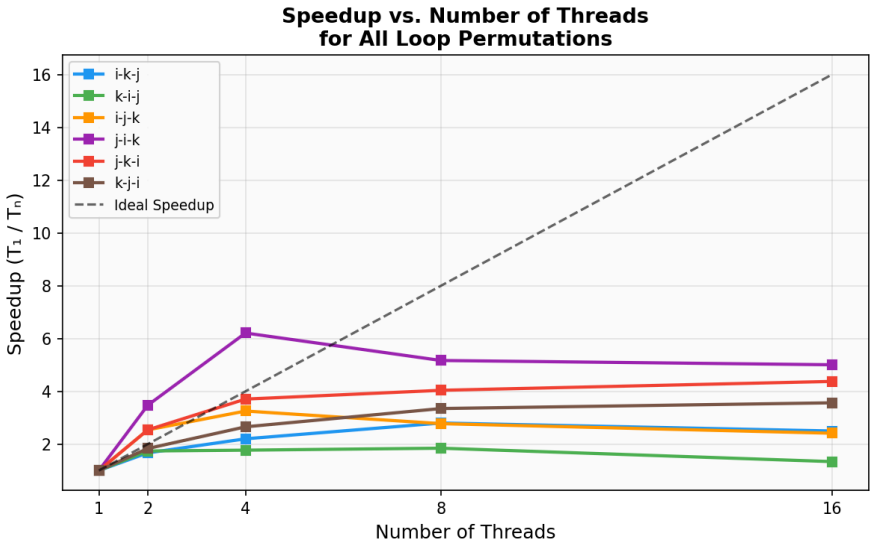

Figure 3: Parallel speedup vs. number of threads. Dashed black line indicates ideal linear speedup. Most permutations peak between 4–8 threads.

3.4 Parallelization Strategy Details

Loop Unfolding (i- and j-outermost): Each thread handles a unique subset of rows or columns. No shared writes occur, so no synchronization is needed. This yields the best parallel efficiency.

Atomic Synchronization (k-outermost): Multiple threads update the same C[i][j] element. The #pragma omp atomic directive serializes these updates, introducing overhead that limits scaling. This is visible in the k-i-j and k-j-i curves, which flatten quickly and even increase at 16 threads.

4. Conclusion

This report confirms that loop permutation order has a dominant effect on matrix multiplication performance - up to 4.5x difference in sequential execution time - primarily driven by cache locality. The i-k-j and k-i-j permutations, where the innermost loop traverses memory sequentially, achieve the best single-threaded and parallel performance.

Parallelization via OpenMP provides significant speedups across all permutations, but with diminishing returns beyond 8 threads. Permutations requiring atomic synchronization (k-outermost) scale poorly due to contention overhead. The most effective strategy combines cache-optimal access patterns (inner loop j) with embarrassingly parallel outermost loops (i or j unfolding), achieving near-ideal speedup at low thread counts.

Ultimately, the 'memory wall' constrains absolute performance: even the fastest permutation is bottlenecked by DRAM bandwidth at large matrix sizes, and no amount of parallelism can fully overcome a fundamentally cache-hostile access pattern.